P, NP, and NP-complete Problems

Travelling Salesman Problem

Input: cities, connections, distances

Output: shortest route visiting each city once

Say input comprises 80 cities.

There is no known algorithm that can find the route in less than millions of years.

All known algorithms take exponential time (in the number of cities) in the worst case.

Types of problems

Optimization problem: construct a solution that maximizes or minimizes some objective function

Decision problem: answer yes/no to a question

Many problems will have decision and optimization versions.

Eg: Travelling salesman problem

- *optimization*: find cycle of minimum weight
- *decision*: Does there exist a cycle of weight < *k?*

The Class *P*

P: the class of *decision* problems that are solvable by (deterministic) algorithms in $O(p(n))$, where $p(n)$ is a polynomial on n

- This class of problems is called *polynomial*.

Why polynomial?

- if not, very inefficient
- nice closure properties: sum and composition of polynomials are always polynomials

The Class *P*

Why **decision problems**?

- exclude problems that have exponentially large output

- many problems that are not naturally decision problems, can be transformed into a series of decision problems.

Non-Deterministic Algorithm

A non-deterministic algorithm takes as input an instance I of a decision problem and:

- Non-deterministic ("guessing") stage: a solution string S is generated. This string *may* be a possible solution of I.
- Deterministic ("verification") stage: a deterministic algorithm uses both I and S as input and outputs **Yes** if the solution S is right; if not it returns **No** or it may not halt

The Class *NP*

NP: the class of decision problems that are solvable by ***n***on-deterministic algorithms in ***p***olynomial time.

Thus ***NP*** can also be thought of as the class of problems whose solutions can be verified in polynomial time.

The solution is a “certificate” that we use to verify efficiently that the problem is in ***NP***.

NP problems

No one has ever found algorithms for any of these problems whose worst-case time complexities are better than exponential...

but no one has ever proved that such algorithms are **not possible**.

No Polynomial-Time Algorithm — No Proof of Its Impossibility

Hamiltonian Circuit: visits each vertex exactly once, return to start.

Travelling Salesman: Cheapest round trip route that visits each city and returns to start

Partition Problem: Given n positive integers, is it possible to partition them into two disjoint subsets with the same sum

Bin Packing: given n items whose sizes are positive rational numbers not larger than 1, put them into the smallest number of bins of size 1

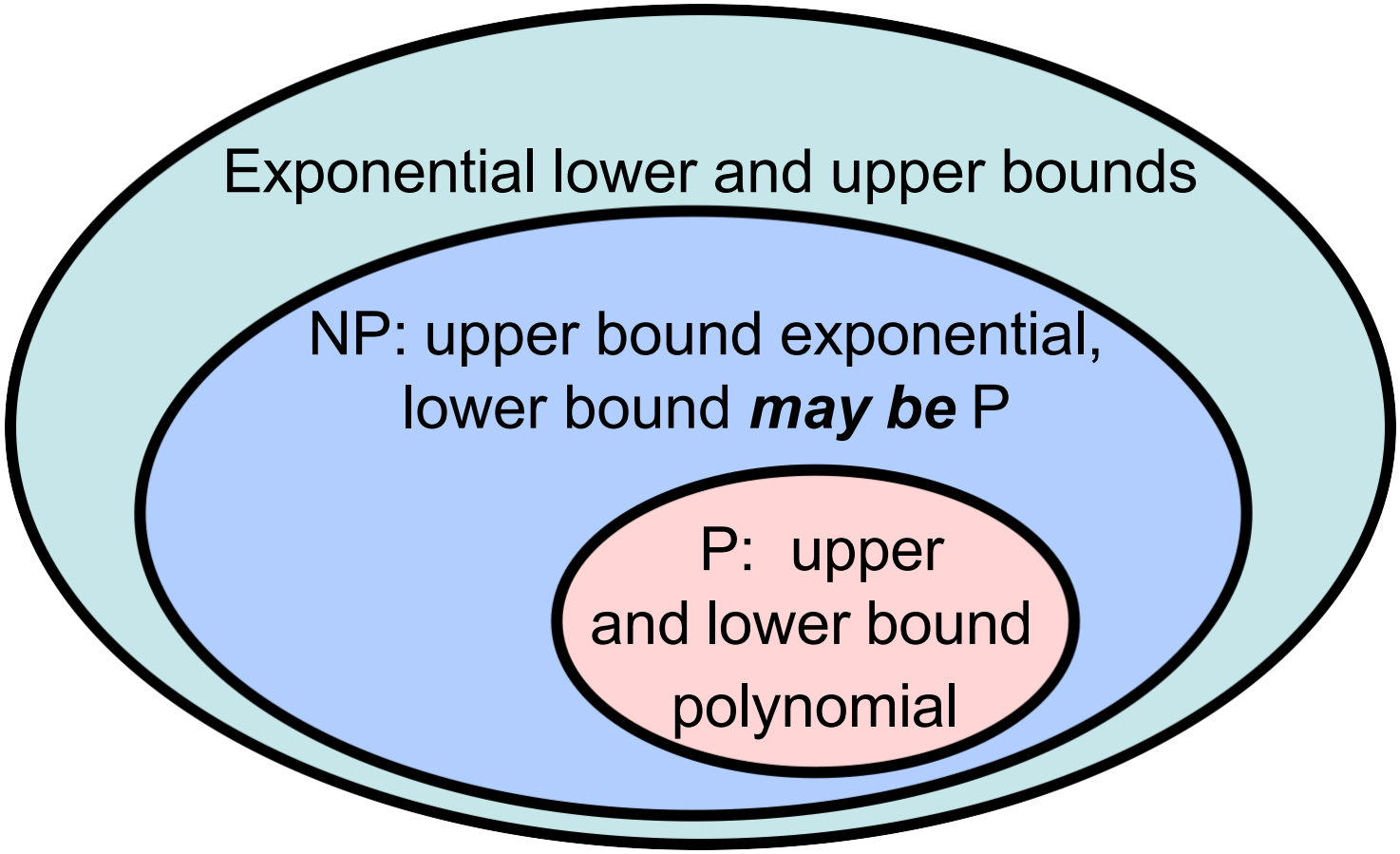
Graph Colouring: find the smallest number of colours that need to be assigned to the graph's vertices so that no two adjacent vertices are assigned the same colour

CNF satisfiability: Given a Boolean expression in conjunctive normal form

is there a truth assignment to the variables that makes the expression true?

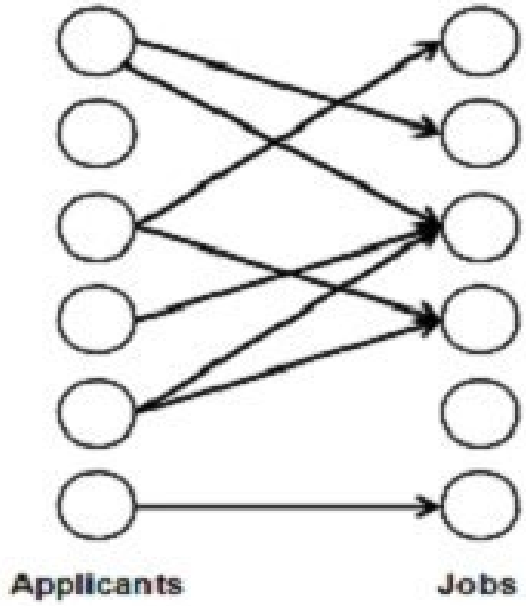
(A OR B OR ~C) **AND** (D OR E) **AND** (~F OR ~G)

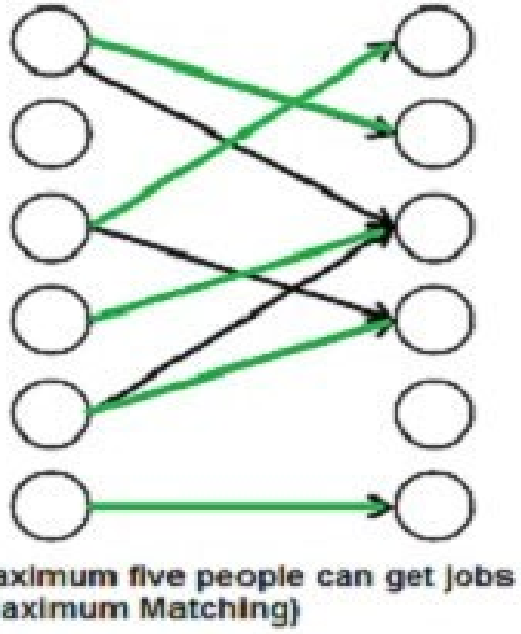
$P \subseteq NP$ (Why? Because if a problem is solvable in polynomial time, then a solution is verifiable in polynomial time)

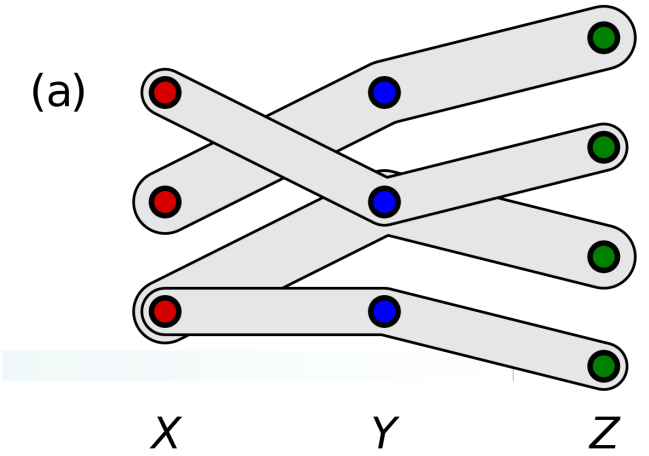


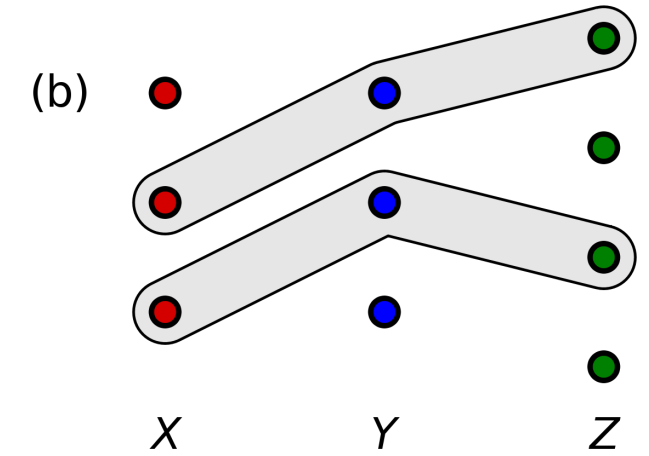
Hard problems, easy problems

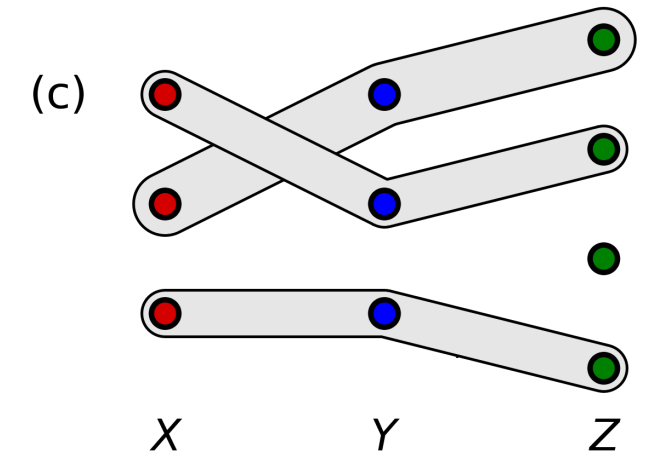
A matching in a Bipartite Graph is a set of the edges chosen in such a way that no two edges share an endpoint. A maximum matching is a matching of maximum size.
In 3D matching, each "edge" comprises 3 points, and again the set is chosen such that no two of them have any point in common (each dot in the picture is in at most 1 such "edge").
Max matching is in class P if bipartite (2D) but only NP if 3D !











the world is full of search problems, some of which can be solved efficiently, while others seem to be very hard.

Hard problems (NP-complete)	Easy problems (in P)
3SAT	2SAT, HORN SAT
TRAVELLING SALESMAN	MINIMUM SPANNING TREE
LONGEST PATH	SHORTEST PATH
3D MATCHING	BIPARTITE MATCHING
KNAPSACK	UNARY KNAPSACK
…	

problems on the right can be solved by algorithms that are specialized and diverse: they are easy for a variety of different reasons.

problems on the left *are all difficult for the same reason! At their* core, they are all the same problem, just in different disguises! They are all *equivalent:* each of them can be reduced to any of the others and back.

P and NP: The Big Question!

Is P a proper subset of NP, or are these two classes in fact the same?

Is P = NP true?

If NP problems are actually *also* solvable in polynomial time, then those two problem sets become one, and P = NP

If NP problems are *not* solvable in polynomial time, then they are not the same, and P NP

S. Cook. The complexity of theorem proving procedures. In *Proc. Third ACM Symp. Theory of Computing* (STOC '71), pages 151-158, 1971

Clay Mathematics Institute offers $1 million prize for proof - http://www.claymath.org/millennium/

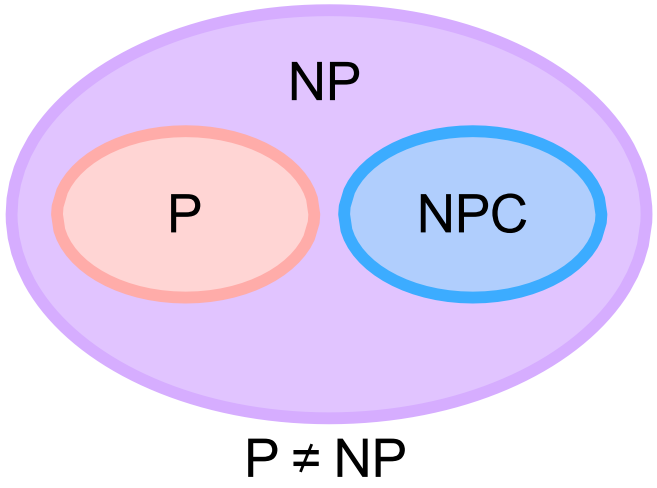
NP-Complete

Is *P* = *NP*?

We have a strong belief that *P* ≠ *NP* which is based on the existence of *NP*-complete problems.

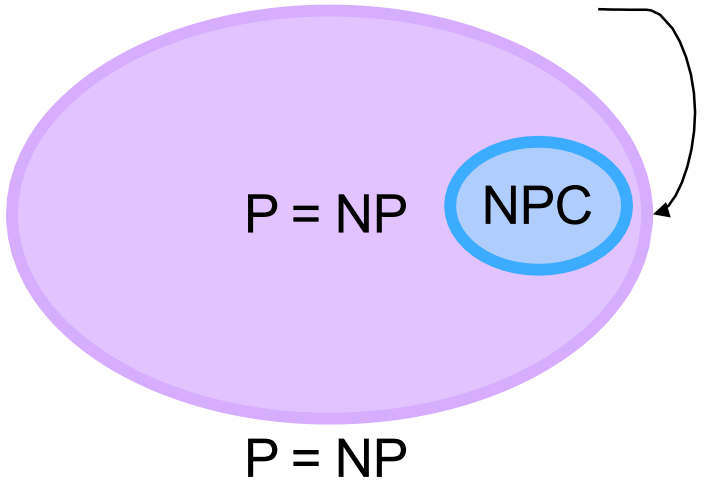
NP-Complete (NPC)

- All problems in NPC have the same degree of difficulty
- **If** any NPC problem can be solved in polynomial time **then all** problems in NP can be solved in polynomial time.



P ≠ NP

Very unlikely*!*



P = NP

Boolean Satisfiability Problem (SAT)

Problem is defined by a Boolean expression written with only **AND(∧, conjunction), OR(∨, disjunction), NOT(¬, negation), variables, and parentheses.**

Given a Boolean expression in CNF, is there some assignment of TRUE and FALSE values to the variables that will make the entire expression true?

- Examples:
  - $(a \vee \neg b) \wedge (\neg a \vee b)$ is satisfiable (set a and b both TRUE)
  - $\neg a \wedge (a \vee b) \wedge (a \vee \neg b)$ is not satisfiable

CNF Expression written as a conjunction (AND) of clauses, where each clause is a disjunction (OR) of literals.

- *k*-CNF if there are at most *k* literals in each disjunction

Satisfiability: CNF SAT

Problem is in NP

Nondeterministic algorithm:

- Guess truth assignment
- Check assignment to see if it satisfies CNF formula

For $(A \vee \neg B \vee \neg C) \wedge (\neg A \vee B) \wedge (\neg B \vee D \vee F) \wedge (F \vee \neg D)$

Truth assignments:

	A	B	C	D	E	F
1.	0	1	1	0	1	0
2.	1	0	0	0	0	1
3.	1	1	0	0	0	1
4.	…			how many more?		

Checking phase: $\Theta(n)$

This example is 3-CNF, as longest disjunction has 3 literals

Polynomially Reducible

A decision problem D_1 is said to be polynomially reducible to a decision problem D_2 if there exists a function *t* that transforms instances of D_1 to instances of D_2 such that:

1. *t* maps all ***yes*** instances of D_1 to ***yes*** instances of D_2 and all ***no*** instances of D_1 to ***no*** instances of D_2 ;

2. *t* is computable by a polynomial-time algorithm.

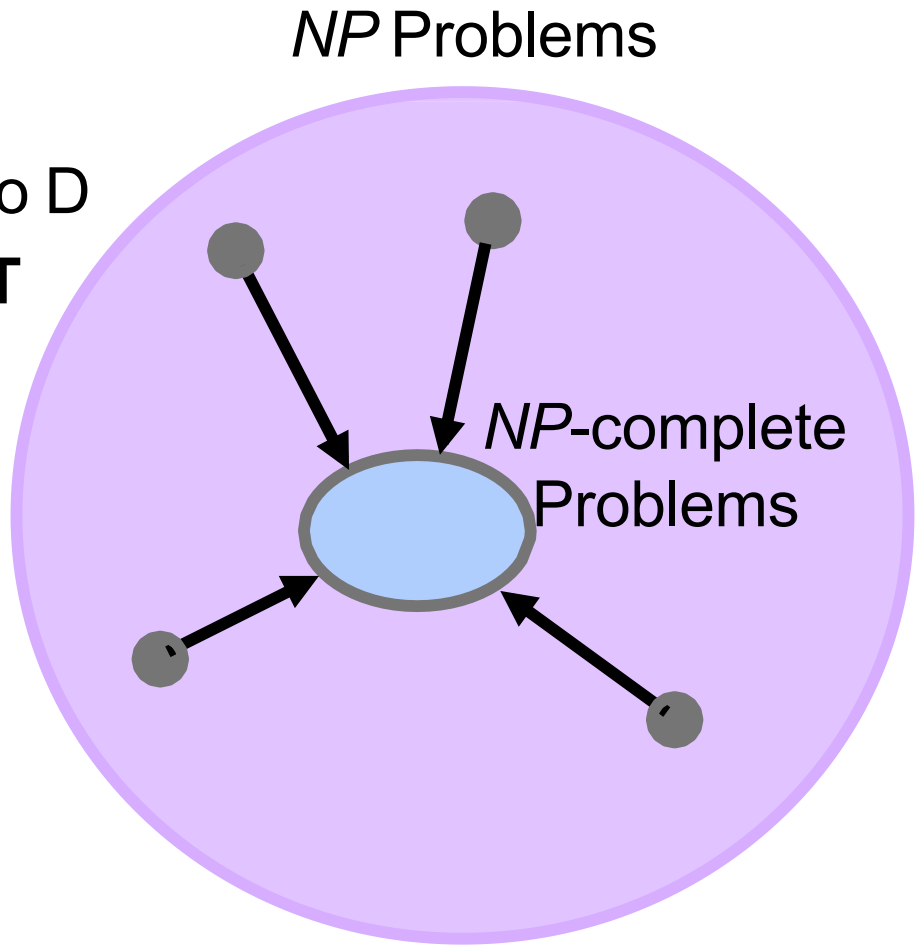
NP-Complete Problems

Decision problem D is NP-complete iff

- D $\in$ NP
- **every problem** in NP is polynomial- time reducible to D

Cook's theorem (1971): **CNF-SAT** is NP-complete

Other NP-complete problems obtained through polynomial-time reductions of known NP-complete problems



Polynomial or worse ?

Close to 1 000 diverse algorithmic problems which all have exponential-time solutions but none have polynomial time ones (e.g. TSP)

No-one has been able to prove that any of them require super-polynomial-time.

If ONE NP-complete problem shown to be P, then P=NP!

NP-Complete Problems

Hamiltonian Circuit: visits each vertex exactly once, return to start.

Travelling Salesman: Cheapest round trip route that visits each city and returns to start

Partition Problem: Given n positive integers, is it possible to partition them into two disjoint subsets with the same sum

Bin Packing: given n items whose sizes are positive rational numbers not larger than 1, put them into the smallest number of bins of size 1

Graph Colouring: find the smallest number of colours that need to be assigned to the graph's vertices so that no two adjacent vertices are assigned the same colour

CNF satisfiability: Given a Boolean expression in conjunctive normal form

is there a truth assignment to the variables that makes the expression true?

Minesweeper NP-Complete

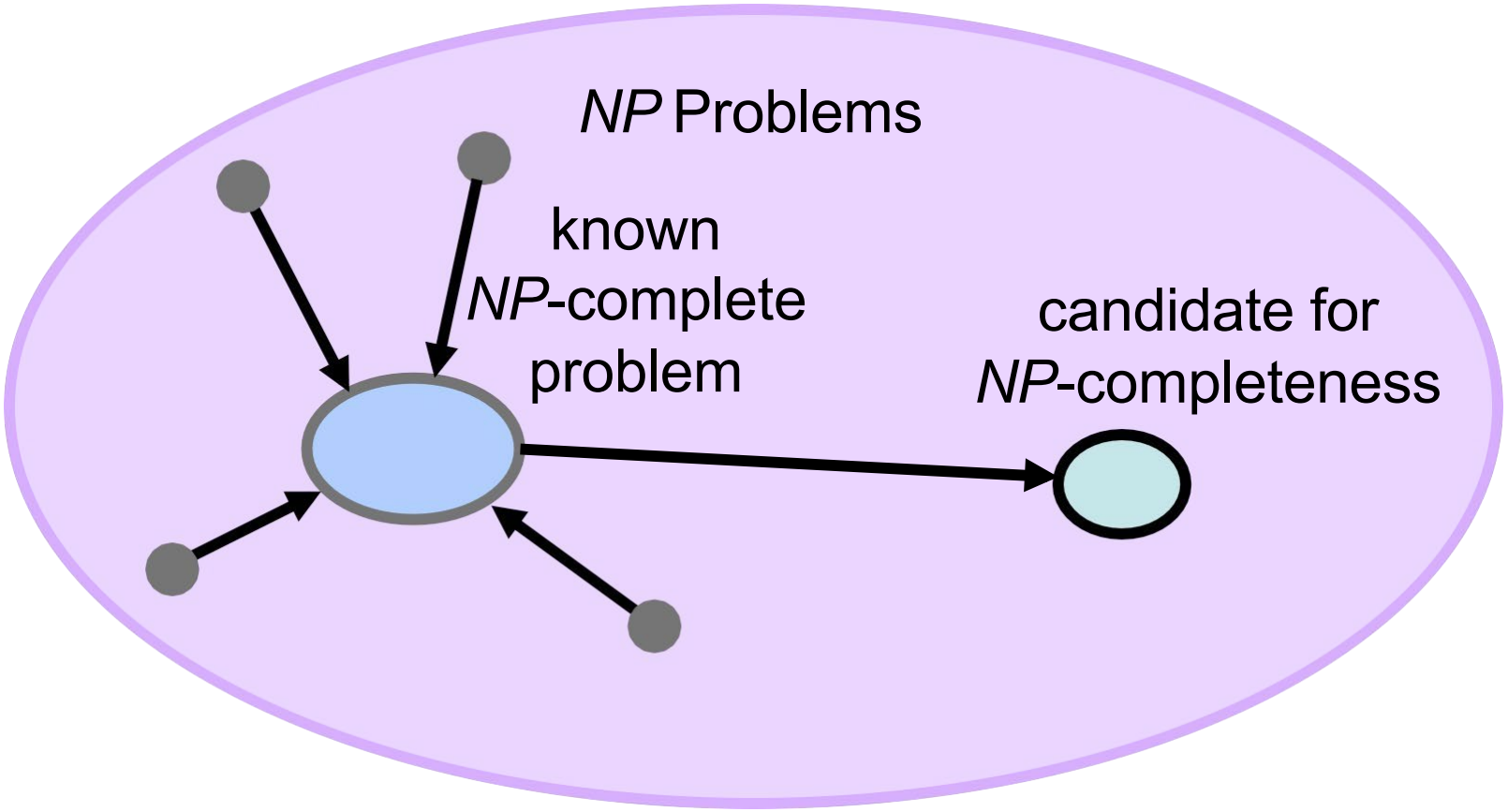
Deciding whether any given display in a minesweeper game is legitimate is NP-Complete.

- Minesweeper Consistency Problem (MCP)
- SAT is reducible to MCP!

See Richard Kaye's Minesweeper Pages: web.mat.bham.ac.uk/R.W.Kaye/minesw/minesw.htm



Proving NP-Completeness by Reduction



To show a problem X is NP-complete

- **Show X is in NP** (i.e. show polynomial time solution verification)
- **Choose a known NP-complete problem Q**
- As Q is NP-complete, all problems in NP α Q
- **Show Q is reducible to X in polynomial time**
- Then all problems in NP reduce to X
- Therefore, X is NP-complete

Hamiltonian Path

A path that passes exactly once through each node of an undirected graph G.

Hamiltonian-Path Problem (HP):

- Given an undirected graph G, decide if G has a Hamiltonian path.

HP is NP-complete.

On the other hand ….

to find if a Eulerian path exists (i.e. one that passes through each edge exactly once), there is a simple test:

if graph is connected and the number of edges incident with any node (except possibly two nodes) is even, then there is a Eulerian path; otherwise, there is not

and … Longest Path Problem (LPP)

Given a graph G with n nodes and e edges, along with an integer K ≤ n, does G contain a simple path (one encountering no node more than once) with k or more edges ?

LPP is **NP - Complete**

Travelling Salesman Problem

Input: cities, connections, distances, cost K

Output: Is there a simple path through each city of cost <= K (Yes / No) ?

Show that TSP is an NP-Complete problem.

Reducing HP to TSP

Given graph G with n nodes, construct a traveling salesman network G^1 as follows:

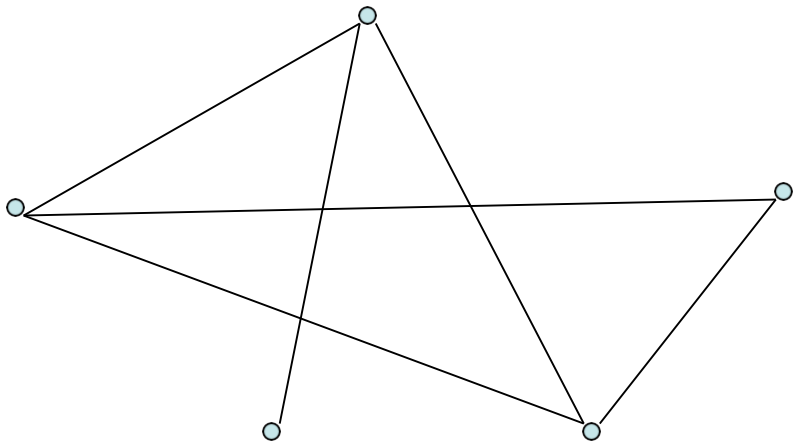
- **the nodes of G^1 are the n nodes of G and**
- **edges are drawn between every 2 nodes,**
- **assigning cost 1 to an edge if it was present in G,**
- **and cost 2 if it was not**

Not difficult to prove

- **G' has a traveling salesman tour of length k iff G has a Hamiltonian path**
- **the transformation takes only a polynomial amount of time**

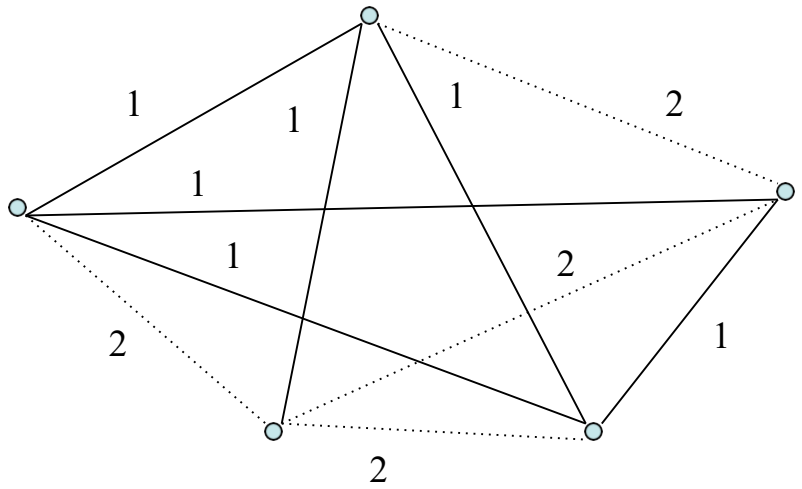
HP example

Is there a simple path that visits each node exactly once ?



HP example reduced to TSP

Is there a travelling salesman tour of cost <= (n + 1) ? (where n is the number of nodes in the graph)



HP α TSP

So:

- **HP is no harder than TSP**
- **TSP is no easier than HP**

So:

if TSP can be solved in polynomial time, then so can HP (because of polynomial – time reduction)

or if HP requires exponential time then so does TSP

Disjoint Paths (DP)

Input: Directed graph G and 2 pairs of distinct nodes (w,x) and (y,z).

Question: Is there a pair of (node-) disjoint directed simple paths in G, one from w to x and the other from y to z ?

DP is known to be NP – complete.

(Aside: DP for undirected graphs can be solved in polynomial time).

Distinguished Edge (DE)

Input: Directed graph G, a pair of distinct nodes *u* & *v*, and an edge *e* in G.

Question: Is there a directed simple path in G from *u* to *v* that passes through *e* ?

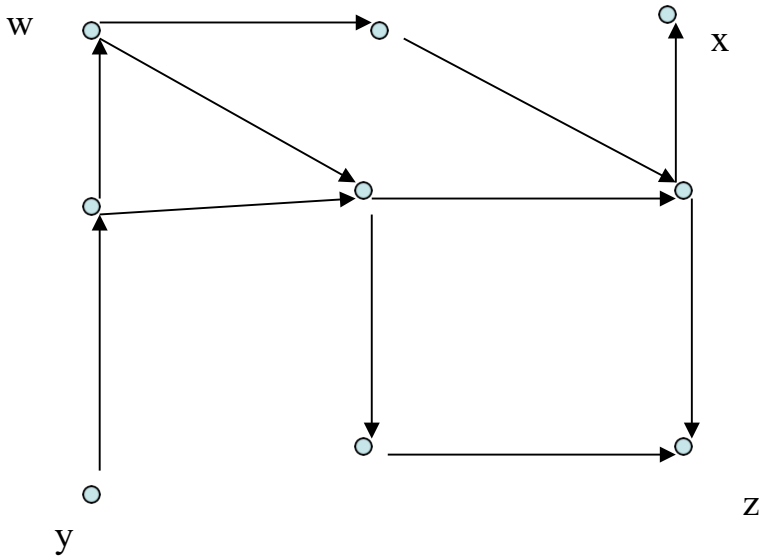
We can reduce DP to DE as follows

(refer to diagrams on the next two slides) :

- **Given input G, (*w*, *x*) and (*y*, *z*) for DP, transform G to G' which is G with edge *e* from *x* to *y*.**
- **Then G has a pair of disjoint simple paths, one from *w* to *x* and the other from *y* to *z*, iff G' has a simple path from *w* to *z* passing through *e*.**

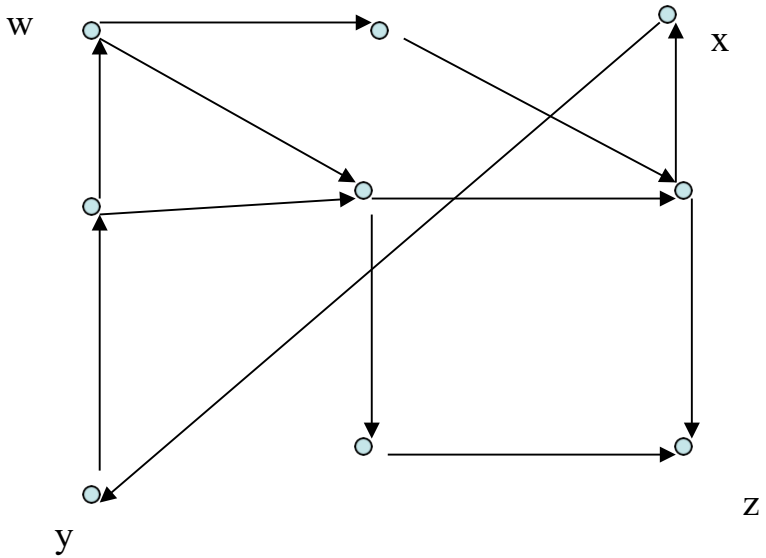
Disjoint Path (DP) Example

Is there a pair of disjoint simple paths, one from w to x and the other from y to z ?



DP reduced to DE

Is there a directed simple path from w to z passing thru edge x->y ?



Directed Hamiltonian Path (DHP)

Directed Hamiltonian-Path Problem (DHP):

- Given a directed graph G, decide if G has a Hamiltonian path (a path that passes exactly once through each node of a directed graph G).

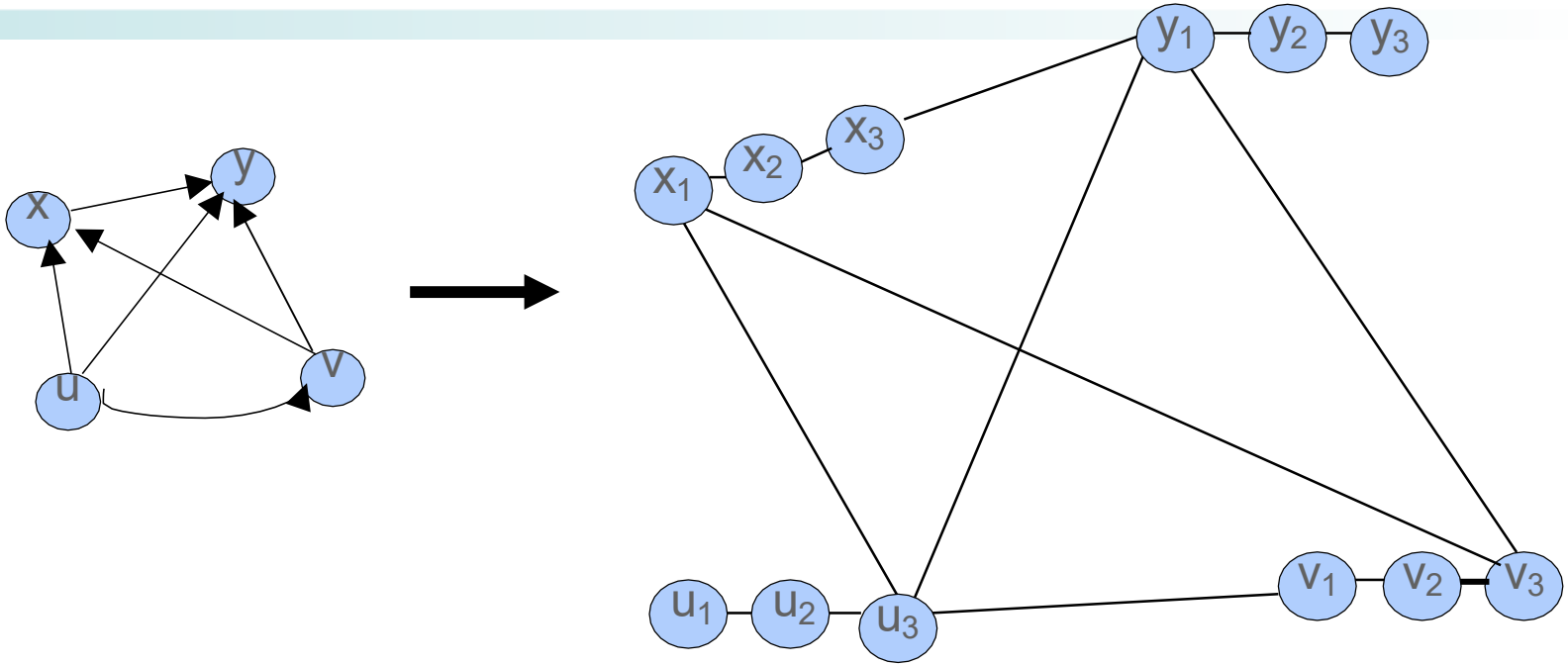
Theorem: DHP is NP-complete.

Undirected Hamiltonian Path (UHP):

Is there a path that passes exactly once through each node of an ***undirected*** graph G?

Show that UHP is NP-complete.

Polynomial-time reduction of Directed Hamiltonian Path to Undirected HP



To show UHP is NP-complete:

1. **Show solution can be verified in P time**
2. **Show known NP-complete problem (namely DHP) is polynomially reducible to UHP (see diagram above).**
3. **Thus UHP is NP-complete.**

Graph isomorphism: in NP, but nobody has shown if it is NP-Complete

Given 2 graphs G1 and G2, is there a one-to-one mapping of the vertices/nodes of G1 to those of G2, such that for all pairs of nodes, there is an edge between those two nodes in G1 if and only if there is an edge between the corresponding two nodes in G2.

In other words, do the 2 graphs have the same structure?

Yes of no (decision problem)

Conclusion

If you can show that any of the problems named as NP-complete are polynomial bounded, fame and fortune are yours.

What do we do about these problems in the meanwhile?

- There are approximation algorithms. (i.e. either their average case is polynomial, or they don't always give the best solution.)